

C# 반복문과 배열 마스터하기

데이터의 흐름을 지휘하는 개발자의 핵심 도구함

복잡한 코드를 직관적인 시각적 설계도로 해독합니다.



반복의 기술: 효율성 vs. 비효율성

⚠ 무식한 방법 (Brute Force)

복사-붙여넣기 기술의 한계. 코드가 길어지고 수정이 어렵습니다.

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("출력");
04     Console.WriteLine("출력");
05     Console.WriteLine("출력");
06     Console.WriteLine("출력");
07     Console.WriteLine("출력");
08 }
```

비효율 & 팽창

🤖 스마트한 자동화 (Automation)

반복문을 사용하면 컴퓨터에게 단 세 줄로 1000번의 작업을 명령할 수 있습니다.

```
01 static void Main(string[] args)
02 {
03     for (int i = 0; i < 1000; i++)
04     {
05         Console.WriteLine("출력");
06     }
07 }
```

효율성 극대화

기본적인 배열 생성 및 메모리 구조

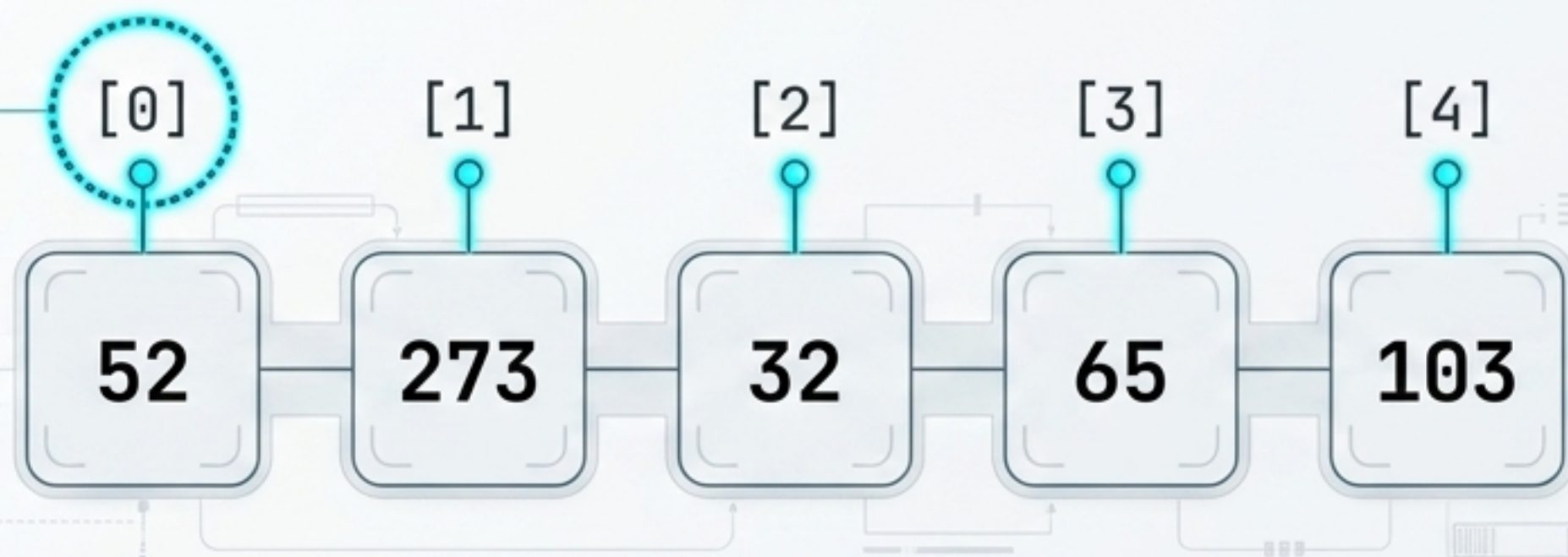
배열 선언 및 초기화

자료형[] 이름 = {자료, 자료}

```
03 int[] intArray = { 52, 273, 32, 65, 103 };
```

메모리 할당 시각화

주의: 인덱스는 항상
0부터 시작합니다.



배열 생성과 메모리 범위 오류

빈 배열 생성 및 길이

```
01 int[] array = new int[100];
```

데이터 없이 100개의 빈 방(기본값 0)을 먼저 예약합니다.

- Length 속성: 배열의 총 방 개수를 알려줍니다.

배열 범위 오류 (Warning: Out-of-Bounds)

```
01 int[] intArray = { 52, 273, 32, 65, 103 };  
07 Console.WriteLine(intArray[5]);
```

0	1	2	3	4	
52	273	32	65	103	비어있음 (Empty)

처리되지 않은 예외: System.IndexOutOfRangeException: 인덱스가 배열 범위를 벗어났습니다.

방이 5개면 인덱스는 4번까지!
5번 방은 존재하지 않습니다.

Toolkit Dashboard



조건 기반 제어

`while, do-while`

이 조건이 참인 동안
계속해!
(언제까지 할 것인가?)



횟수 기반 제어

`for`

정확히 N번만 반복해!
(몇 번 할 것인가?)



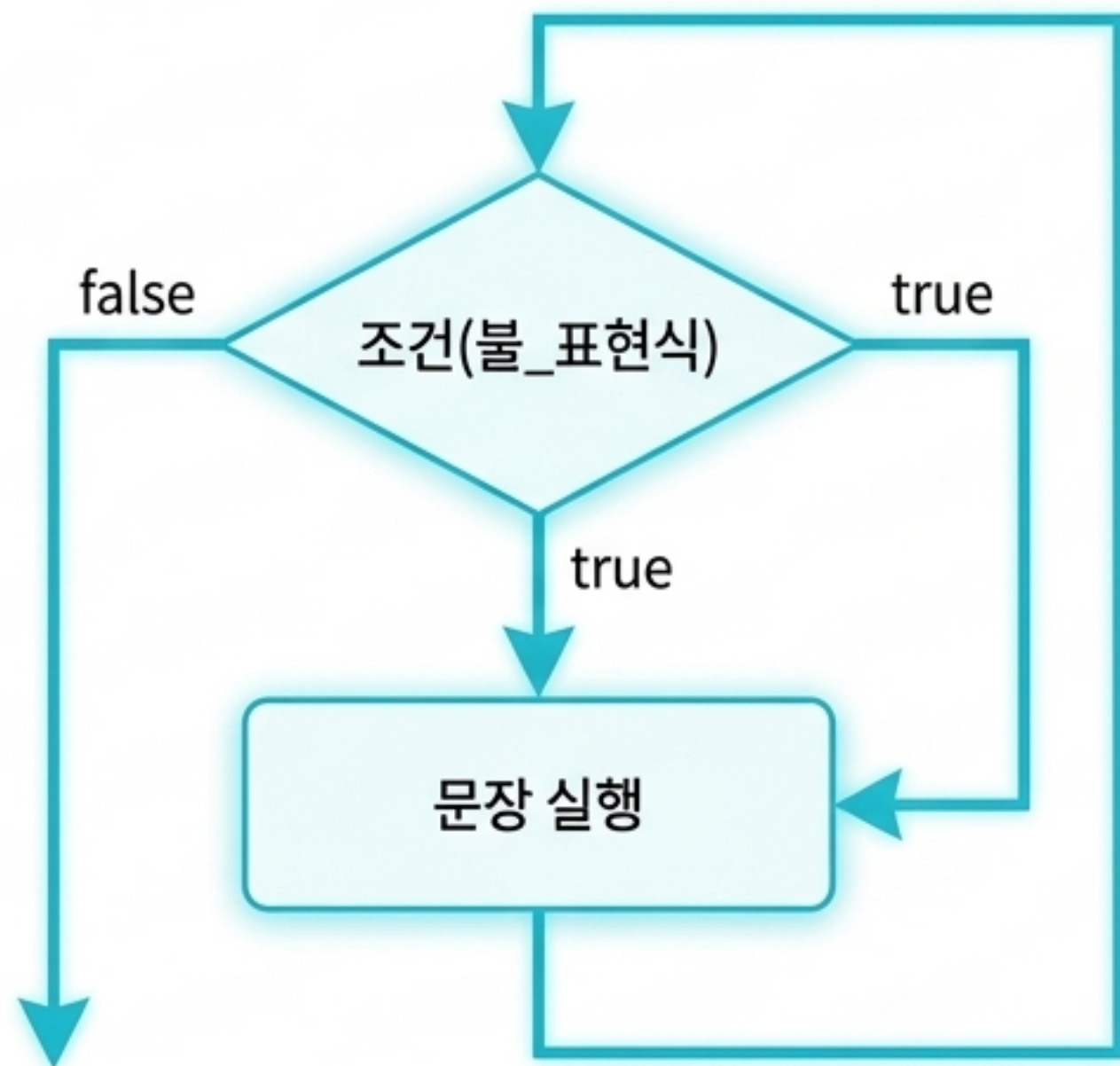
컬렉션 기반 제어

`foreach`

주머니 안의 모든 요소를
하나씩 꺼내!
(무엇을 꺼낼 것인가?)

while 반복문 핵심 로직 및 실전 활용

while 반복문 핵심 로직 (Core Logic)



엣지 케이스 및 실전 활용 (Edge Cases & Practical Usage)

무한 반복 (Infinite Loop Warning) ⚠

```
01 static void Main(string[] args)
02 {
03     while (true)
04     {
05         Console.WriteLine("무한 반복");
06     }
07 }
```

탈출 조건이 없으면
영원히 갇힙니다.

시간 경과를 통한 반복문 이탈 (Time-Based Break) 🕒

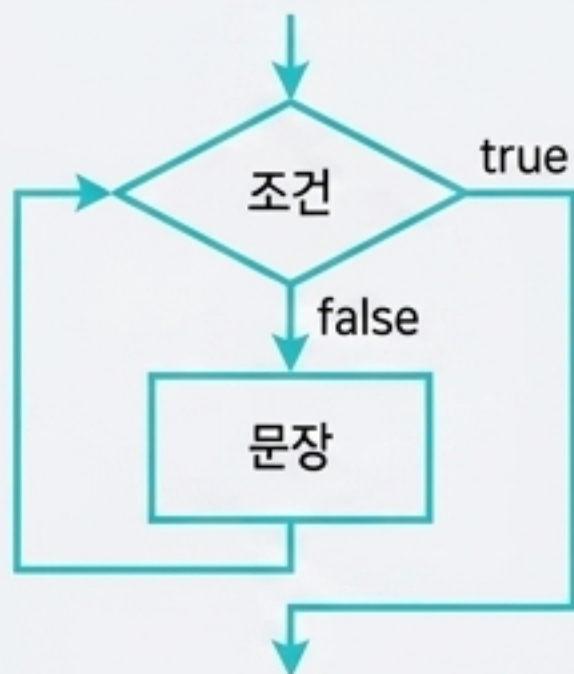
```
01 static void Main(string[] args)
02 {
03     long start = DateTime.Now.Ticks; // 변수를 선언합니다.
04     long count = 0;
05
06     // 반복을 수행합니다: 10000000Tick = 1초
07     while (start + (10000000) > DateTime.Now.Ticks)
08     {
09         count++;
10     }
11
12     Console.WriteLine(count + "만큼 반복했습니다."); // 출력합니다.
13 }
```

반복 횟수가 불명확할
때(예: 시간 경과) 강력함.

while

질문 먼저, 행동은 나중에 (사전 검사)

조건이 처음부터 거짓이면 단 한 번도 실행되지 않음.

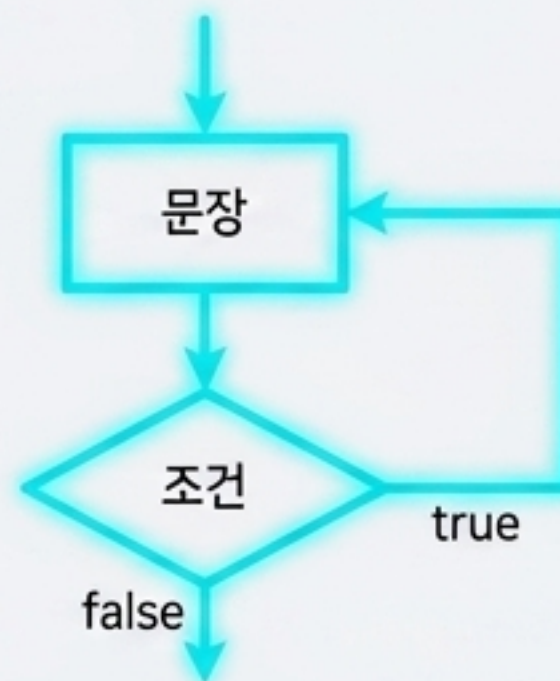


```
while (조건) {  
    // 실행될 문장  
}
```

do-while

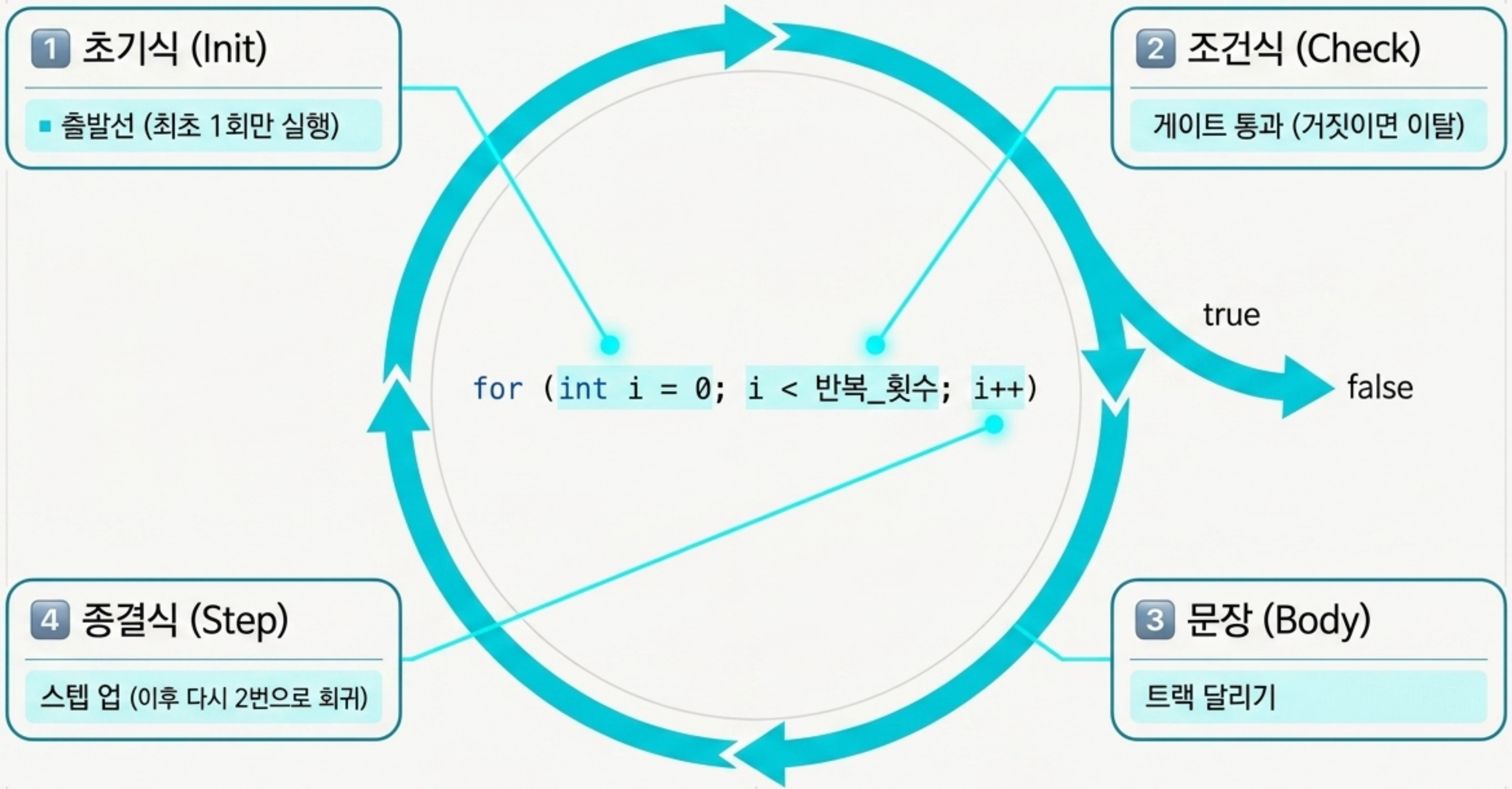
일단 행동, 질문은 나중에 (사후 검사)

문장을 최소 1회 실행 보장.



```
static void Main(string[] args)  
{  
    string input;  
    do  
    {  
        Console.Write("입력(exit를 입력하면 종료): ");  
        input = Console.ReadLine();  
    } while (input != "exit");  
}
```

최초 1회 무조건 실행



한계 돌파 - 곱셈과 자료형

```
01 static void Main(string[] args)
02 {
03     // 변수를 선언합니다.
04     long output = 1;
05
06     // 반복을 수행합니다.
07     for (int i = 1; i <= 20; i++)
08     {
09         output *= i;
10     }
11
12     // 출력합니다.
13     Console.WriteLine(output);
14 }
```

● 곱셈은 초기값이 0이면 안 됩니다.

● 결과값이 int 한계(약 21억)를 가볍게 돌파하므로 더 큰 그릇(long)이 필요합니다.

문자열의 숨겨진 얼굴

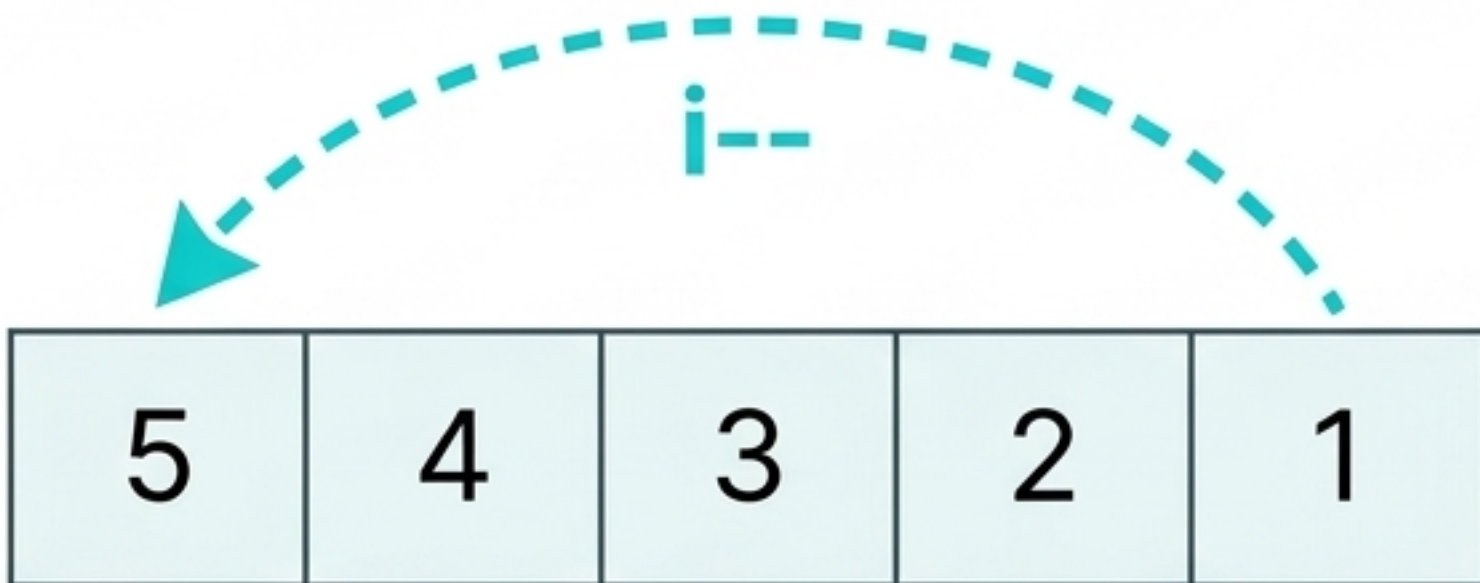
```
01 static void Main(string[] args)
02 {
03     for (int i = '가'; i <= '힉'; i++)
04     {
05         Console.Write((char)i);
06     }
07 }
```

● char 자료형은 내부적으로 숫자입니다.
가(44032)부터 힉(55203)까지 11,172자를
숫자로 취급하여 1씩 증가시키며 반복할 수 있습니다.

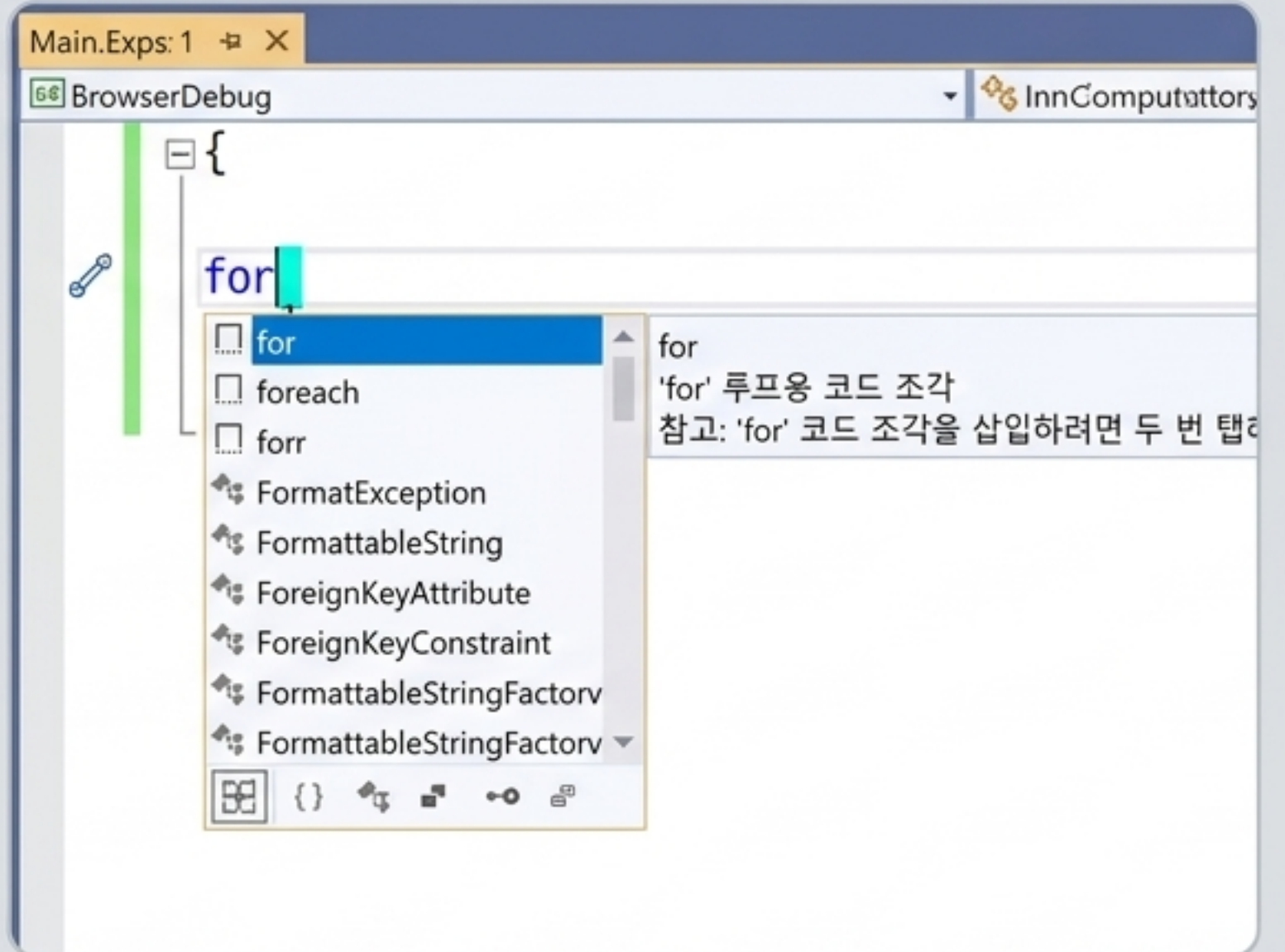
역 for 반복문

```
for (int i = length - 1; i >= 0; i--)  
{  
    // 반복 실행 코드  
}
```

인덱스의 끝($\text{Length} - 1$)에서 시작하여
0까지 거꾸로($i--$) 카운트다운.



⚡ Productivity Hack



VS에서 'for'를 입력하고 [Tab] 키를 두 번
누르면 뼈대 코드가 자동 완성됩니다.

무거운 문법 (Heavy)

```
for (int i = 0; i < 컬렉션.길이; i++)  
{  
    자료형 변수 = 컬렉션[i];  
}
```

인덱스 변수를 만들고, 길이를 체크하고,
배열에서 요소를 다시 꺼내야 하는 번거로움.

우아한 순회 (Elegant)

```
foreach (string item in array)  
{  
    // 출력합니다.  
    Console.WriteLine(item);  
}
```

배열(컬렉션) 안의 요소를 처음부터
끝까지 알아서 하나씩 꺼내옵니다.

Pro Tip

var 키워드: 자료형을 명시할 필요 없이
컴파일러가 알아서 타입을 추론하여
코드를 더 간결하게 만듭니다.

차원의 개념

1차원 (반복문 1겹): 선



2차원 (반복문 2겹): 면



별 피라미드 코드

<IMAGE_0>

'i' 값에 1번에 줄 번호를
채경 변경

```
for (int i = 0; i < 10; i++)  
{  
    for (int j = 0; j < i + 1; j++)  
    {  
        Console.WriteLine('*');  
    }  
    Console.WriteLine('\n');  
}
```

줄 (Row) 변경

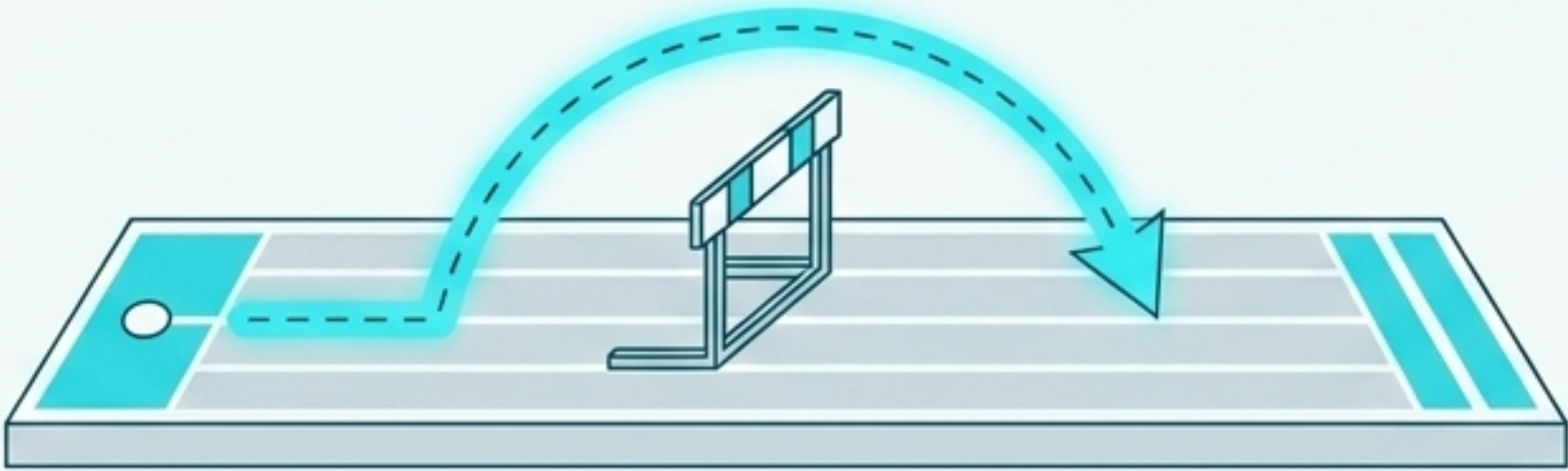
칸 (Column) 채우기

'j' 이록하는 'i+1' 간에
칸 코문캄을 채우기

Bonus: 역방향 피라미드

```
<IMAGE_1>  
for (int i = 0; i < 10; i++)  
{  
    for (int j = 0; j < 10 - i; j++)  
        Console.WriteLine(' ');  
    for (int j = 0; j < i + 1; j++)  
        Console.WriteLine('*');  
    Console.WriteLine('\n');  
}
```

The Jump (continue Flow)



continue: 반복문을 완전히 종료하지 않고, 현재 반복만 멈춘 뒤 다음 반복으로 점프합니다.

```
01 static void Main(string[] args)
02 {
03     for (int i = 1; i < 10; i++)
04     {
05         if (i % 2 == 0)
06         {
07             continue; // Skips to the next iteration
08         }
09         Console.WriteLine(i); // Executed only for odd numbers
10     }
11 }
```

짝수라면 다음 반복으로 바로 넘어갑니다.
따라서 이 아래의 코드(09행)는 실행되지 않습니다.

Clean Code Refactoring (Inverting Logic)

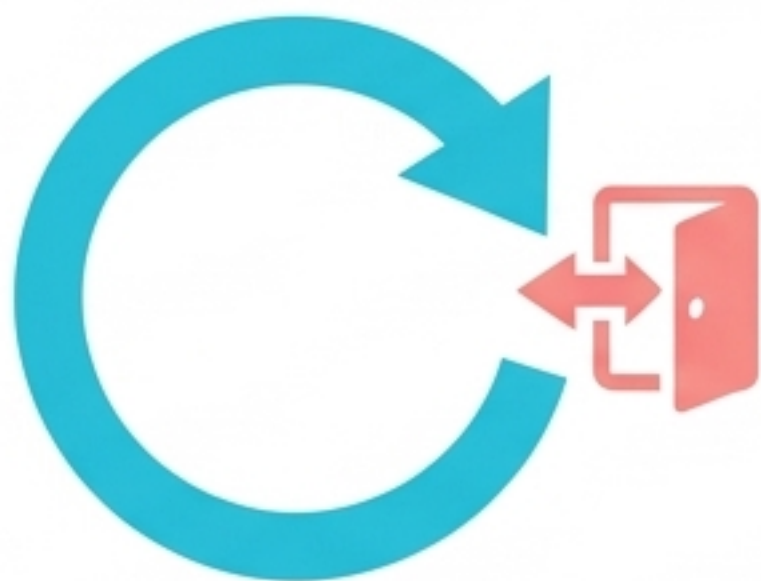
Refactoring: Inverting Logic

조건 뒤집기: 더 깔끔한 코드 구조.

Inverting the condition removes the need for 'continue', resulting in cleaner, more direct code.

```
01 static void Main(string[] args)
02 {
03     for (int i = 1; i < 10; i++)
04     {
05         if (i % 2 != 0) // Condition inverted to check for odd numbers
06         {
07             Console.WriteLine(i);
08         }
09     }
10 }
```

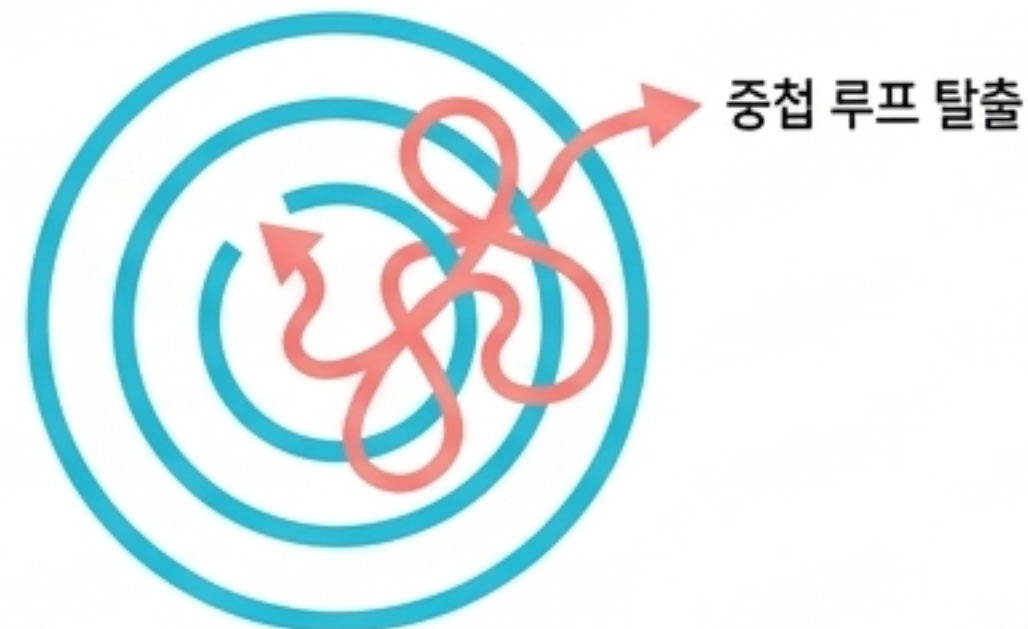
break - 단일 탈출



```
01 while (true)
02 {
03     Console.WriteLine("숫자를 입력해주세요 (짝수 종료): ");
04     int input = int.Parse(Console.ReadLine());
05     if (input % 2 == 0)
06     {
07         break; // 현재 반복문 탈출
08     }
09 }
```

현재 속해 있는 가장 가까운 반복문 1개를 즉시 파괴하고 밖으로 나갑니다.

goto - 다중 탈출



```
01 for (int i = 0; i < 10; i++)
02 {
03     Console.WriteLine("외부 반복문");
04     for (int j = 0; j < 10; j++)
05     {
06         Console.WriteLine("내부 반복문");
07         goto doNotUse; // 중첩 루프 탈출
08     }
09 }
10 doNotUse:
11     Console.WriteLine("goto 키워드 실행");
```

주의: 여러 겹을 한 번에 벗어날 수 있지만, 코드의 흐름을 꼬이게(스파게티 코드) 만들 수 있으므로 가급적 사용을 피하세요.

The Loop Decision Matrix

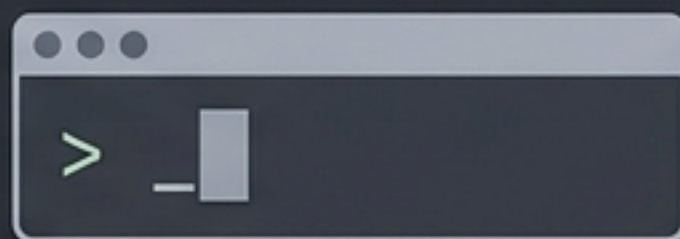
[키워드]	[목적]	[특징]	[Best For]
for	명확한 횟수 반복	초기식/조건식/종결식이 한눈에 보임	수학적 계산, 인덱스 기반 배열 접근.
while	유연한 조건 통제	조건이 참인 동안 무한 반복 가능	사용자 입력 대기, 외부 요인(시간) 대기.
foreach	컬렉션 순회	인덱스 없이 요소만 쫓쫓 뽑아냄	배열, 리스트 내의 모든 데이터를 처음부터 끝까지 읽을 때.

상황에 맞는 최적의 반복문을 선택하는 것이 클린 코드(Clean Code)의 시작입니다.

Project > src > TitleSlide.cs

C# Developer's Toolkit

문자열 처리부터 인터랙티브 콘솔 애니메이션까지,
동작하는 애플리케이션을 조립하는 실전 가이드



가장 흔한 실수: 문자열의 불변성 (Immutability)

C#의 모든 문자열 처리 메서드는 자신을 변경하지 않고 새로운 복사본을 반환합니다.



```
static void Main(string[] args)
{
    string input = "Potato Tomato";
    input.ToUpper();
    Console.WriteLine(input);
}
```

원본은 변하지 않음!

실행 결과

POTATO TOMATO
potato tomato

텍스트 가공 도구 상자 (String Transformation)

대소문자

"Potato Tomato"



ToUpper() / ToLower()



"POTATO TOMATO" /
"potato tomato"

문자열의 영문자를 모두 대문자
또는 소문자로 변환합니다.

공백 제거

" test \n"



Trim()



"test"



문자열 양 옆의
공백 제거

TrimStart() (앞 공백 제거),
TrimEnd() (뒤 공백 제거)

자르기
(배열로)

"감자 고구마 토마토"



Split(' ')



["감자", "고구마",
"토마토"]



특정 문자 또는
문자열로 잘라
배열로 만듭니다.

결합하기
(문자열로)

["감자", "고구마",
"토마토"]



string.Join(",", array)



"감자,고구마,토마토"



배열의 요소를 뭉쳐
문자열로 변경합니다.

시각적 제어 도구 (Console Control Tools)



`Console.Clear()`

콘솔 화면의 모든 글자를
즉시 지웁니다.



`Console.SetCursor
Position(x, y)`

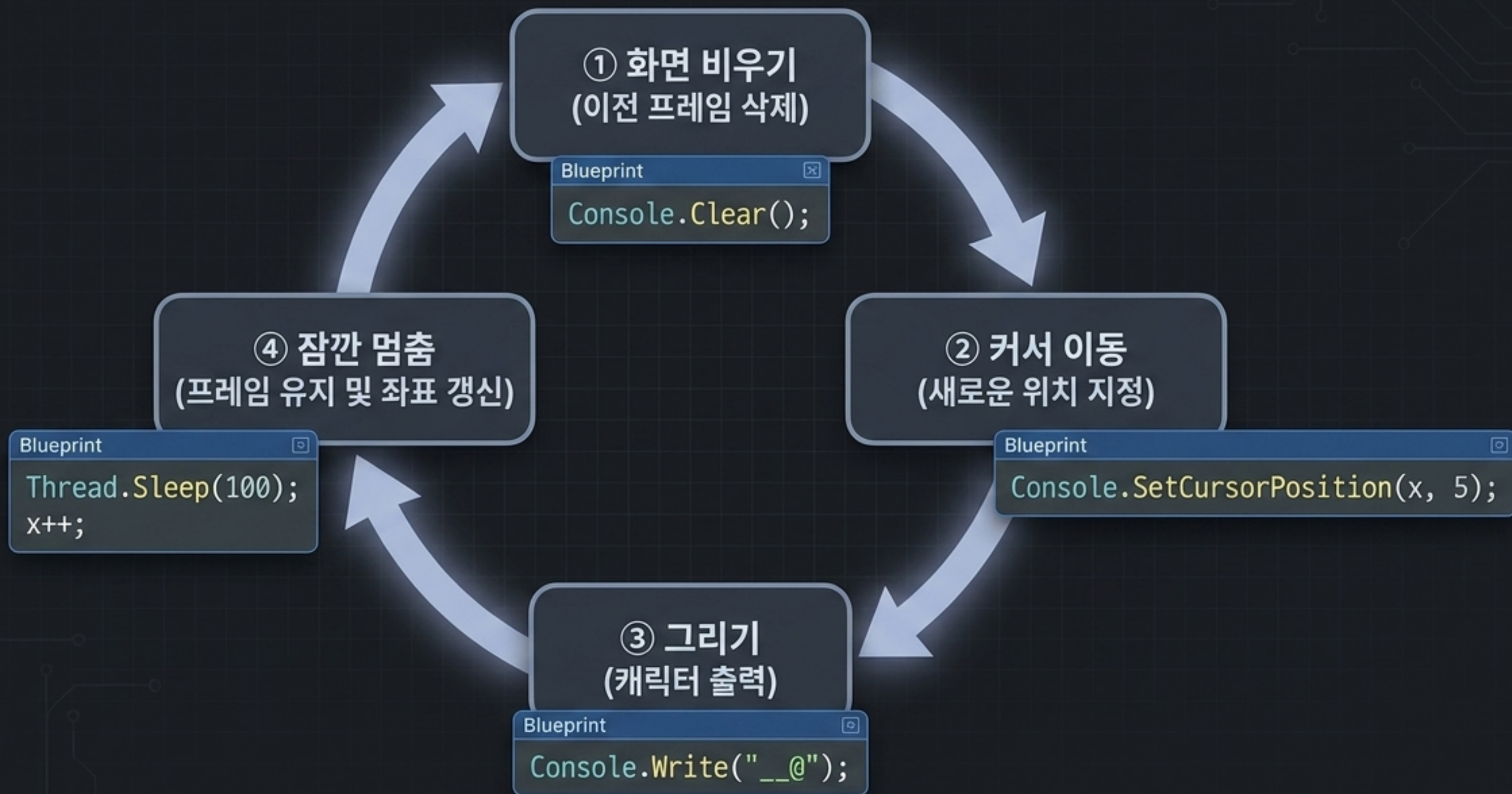
원하는 위치로 출력 커서를
순간이동 시킵니다.



`Thread.Sleep(ms)`

입력한 밀리초(ms) 단위만큼
프로그램의 시간을 멈춥니다.

애니메이션의 원리: 화면 갱신 루프 (Animation Loop)



디테일 렌더링: 꿈틀거리는 달팽이

```
if (x % 3 == 0) Console.Write("__@");  
else if (x % 3 == 1) Console.Write("_^@");  
else Console.Write("^__@");
```

Frame 1 (x=1)

_ ^ @

Frame 2 (x=2)

^ __ @

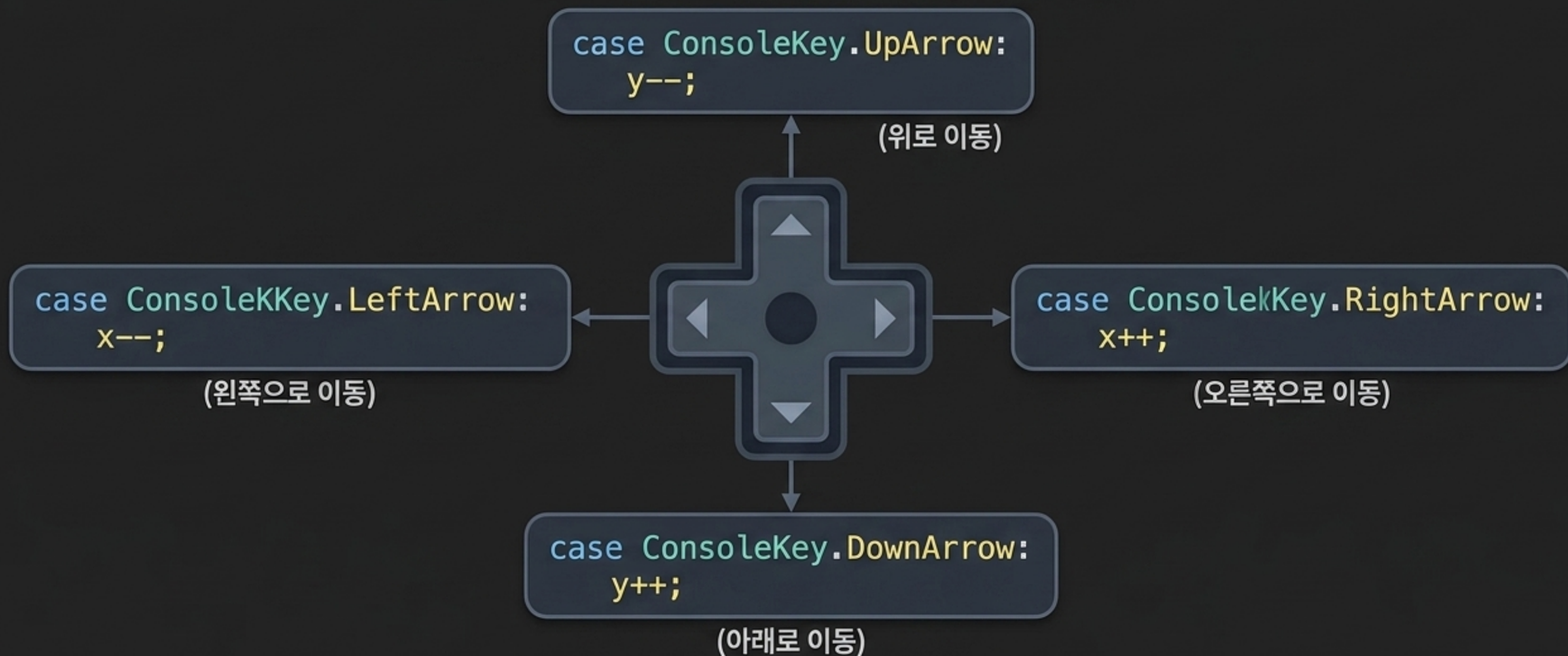
Frame 3 (x=3)

__ @

변수 x의 증가에 따라 달팽이의 모양이
3단계를 거치며 순환합니다.

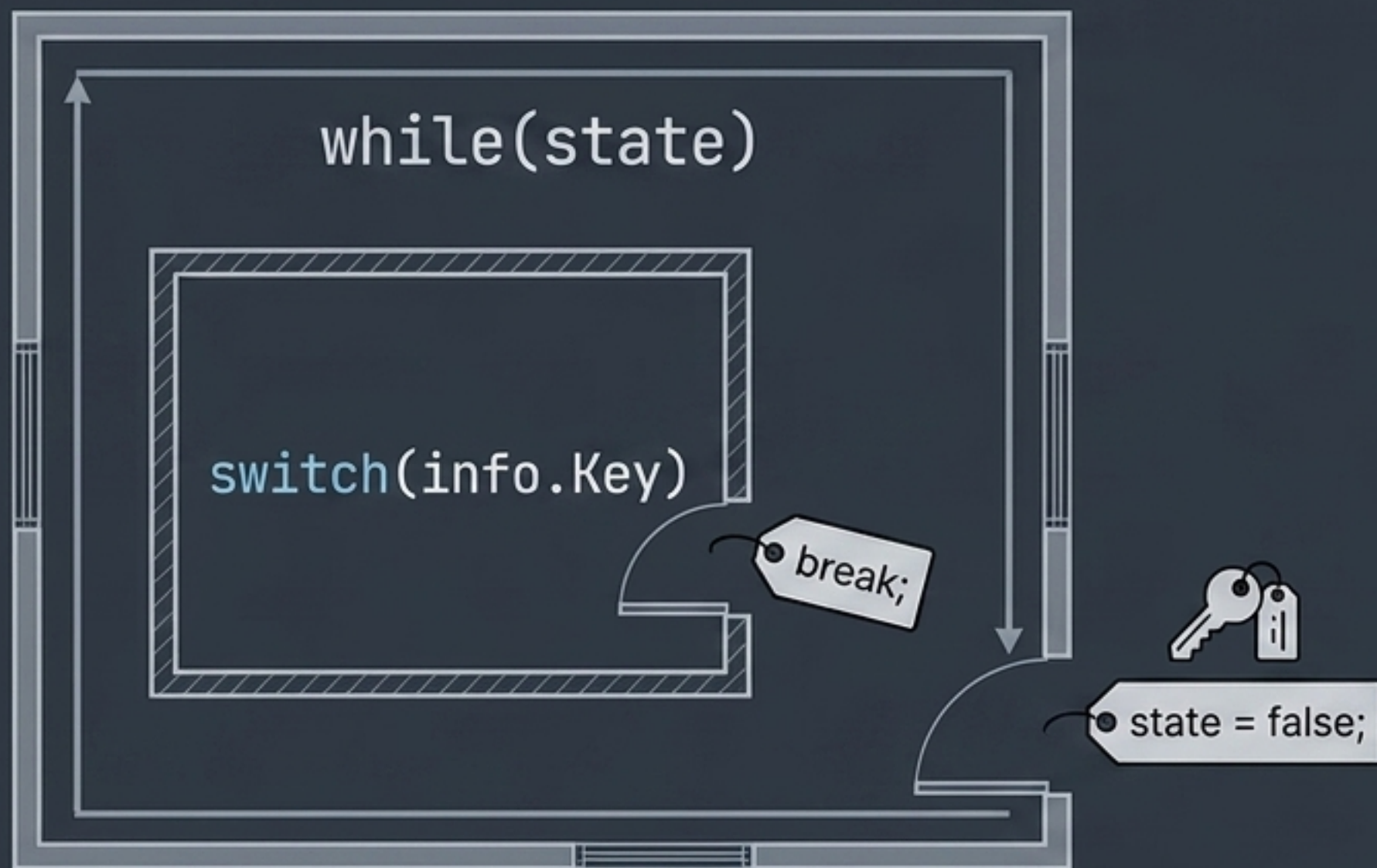
인터랙티브 입력: 방향키로 제어하기

`while (true)` 무한 반복문 안에서 `Console.ReadKey()`로 입력을 대기합니다.



아키텍처: 탈출 스코프 (Escape Scope)

switch 문 내부의 break는 조건문(안쪽 방)만 빠져나갑니다. 무한 반복문(전체 집)을 완전히 종료하려면 제어 변수(bool state)의 상태를 변경해야 합니다.



```
case ConsonkeyKey.X:  
    state = false; // 마스터키 작동  
    break;         // 방문 열기
```

Pro-Tip: 화살표(-->) 연산자의 착시 효과

C#에는 --> 라는 화살표 연산자가 존재하지 않습니다. 이것은 후위 감소 연산자(--)와 크다 비교 연산자(>)가 결합된 시각적 착시입니다.

코드를 실행하면 i가 0이 될 때까지 1씩 줄어들며 반복을 수행합니다. 띄어쓰기가 코드의 논리를 바꾸지 않는다는 것을 보여주는 흥미로운 트릭입니다.

```
static void Main(string[] args)
{
    int i = 10;
    while (i --> 0)
    {
        Console.WriteLine(i);
    }
}
```



while ((i--) > 0)